

Browsing Large Graphs with Tile Pyramids and Sleeve Routing in the Browser

Category: System

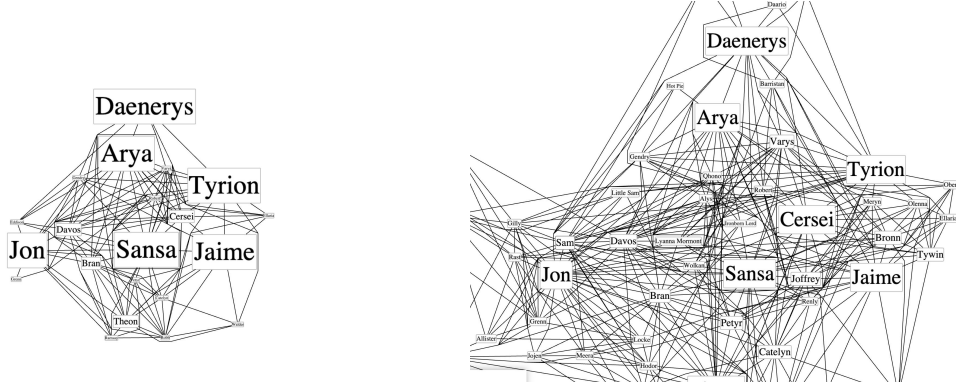


Figure 1: Two adjacent pyramid levels of the Game of Thrones graph rendered by the WebGL renderer of GJS with the sleeve router. The coarser level is on the left, the finer one on the right; both panels are cropped to the same world window. For each level a new graph is assembled from a PageRank-ranked prefix of the nodes, and edges are routed against the obstacles introduced at that level.

ABSTRACT

We present a new way to visualize a large graph in the style of online geographic maps. The method builds a *tile pyramid* for *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on those maps.

The edges are routed by a method we call *sleeve routing*, which searches the dual graph of a Constrained Delaunay Triangulation to select a sequence of triangles through the free space, then applies the funnel algorithm to compute a shortest path inside the selected sleeve. We apply several heuristics to speed up the routing.

We implemented our approach in the WebGL renderer of GJS (a placeholder name used during double-blind review), an open-source TypeScript library for graph visualization in web browsers, with the entire pipeline running client-side, without a dedicated server. Our benchmark suite contains nine graphs with up to 32,768 nodes and 236,978 edges, and measures browser-side parsing, layout, routing, and tile-pyramid construction. The renderer’s demo can be seen at <https://gjs-sleeve.surge.sh/>; the sources and packages are described under Supplemental Materials.

Index Terms: Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL.

1 INTRODUCTION

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,¹ and users expect the smooth pan-and-zoom experience familiar from online maps. GJS is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme with semantic zoom.

GJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [2]. It supports the layered Sugiyama scheme [46], Pivot MDS [15], and IPSep-CoLa [22], with edges routed around node obstacles. Throughout this paper

our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The graph loading pipeline builds a hierarchy of tiles—analogueous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers` package, giving smooth pan and zoom. A live demo of the WebGL renderer is available online².

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *sleeve routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 RELATED WORK

Graph visualization tools. Graphviz [26, 5] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [7]; it does not support tiling or WebGL rendering. MSAGL [41] is a desktop layout engine that did not have the features described here. On the web, Sigma.js [8] and Cytoscape.js [24] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [6] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [1] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [40] introduced the tile-pyramid and node-ranking scheme, but it runs only on Windows, ships raster tiles, and reuses one global edge routing that is merely clipped per tile;

¹In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2³² bytes).

²<https://gjs-sleeve.surge.sh/>

GJS brings the idea to the browser with vector tiles and per-level sleeve routing. Cornac [43] handles huge graphs with tiles and edge bundling [35, 33] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [12], which organizes nodes into a hierarchical cluster tree, and topological fish-eye views [25], which distort a multilevel layout [28, 34] under the focus; neither targets browser-side tile pyramids or per-level edge routing. Hierarchical edge bundling [32] and force-directed bundling [33] are complementary clutter-reduction techniques applied *after* edges have been routed.

Shortest paths and edge routing. Our sleeve algorithm walks the dual of a Constrained Delaunay Triangulation [45] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [38, 16, 27, 4]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [31] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [21] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [19] but requires a simple containing polygon. yWorks [11] offers “Organic edge routing” based on force-directed simulation. The approach of Dwyer and Nachmanson [23] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott, and Stuckey performs obstacle-avoiding connector routing with incremental updates [47].

CDT-based pathfinding. The closest precedent to our routing pipeline is the CDT-and-funnel pathfinder of Demyen and Buro [18], originally designed for game-AI units of nonzero radius. They run admissible A* search on the CDT dual (with a reduced-graph variant that collapses corridor chains into single abstract edges) to recover provably shortest paths. Their search is, however, per-query: the bounds depend on the start and goal, so it cannot share work across the edges with the same source as our per-source Dijkstra does. For the graphs we target this would be roughly an order of magnitude slower; we therefore trade their proven optimality for speed.

3 TILING

The input to tiling is a laid-out graph together with routed edges: we prefer to route the edges with sleeve routing, Section 4, to achieve visual stability across level changes. The output is a sequence of $Z+1$ levels: level Z is the finest, and level 0 is the coarsest. Each level is a collection of tiles indexed by integer (x, y) grid coordinates.

A tile is a rectangular viewport together with its *tile data*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a curve confined to the tile rectangle, paired with the non-empty list of graph edges whose routes enter and exit the tile at very close endpoints. Bundles usually capture the stretches on which several routes travel close together inside a common sleeve; we represent each such stretch as a single clip with an attached edge list, rather than as one clip per edge.

Pseudocode 1 describes the algorithm building the tile pyramid.

How the index of the finest layer, Z , is chosen. We treat the tiles differently when calculating Z and when preparing for rendering. There is no element filtering in the former case: all graph entities are represented in each level. We continue increasing Z , generating new levels, and splitting tiles, until a stopping condition is met. Then, while keeping the finest level intact, we rebuild the tiles of levels $0, \dots, Z-1$ for rendering.

We start from $Z = 0$ and grow the pyramid one level at a time. We stop as soon as any of three conditions is met:

1. every tile at the new level contains at most $\mathcal{C}$ elements, with default $\mathcal{C} = 500$;

Algorithm 1 BUILD_TILE_PYRAMID($G, \text{TileCapacity}, \text{minTileSize}, \mathcal{M}_{\max}$)

```

1:  $T_0 \leftarrow$  single tile whose rectangle is the smallest axis-aligned
   square of side  $2^{\lceil \log_2 \max(w, h) \rceil}$  centered on  $G$ 's bounding box of
   width  $w$  and height  $h$ , holding all of  $G$ 
2:  $\text{levels} \leftarrow [T_0]$ ;  $z \leftarrow 0$ ;  $\text{used} \leftarrow |T_0| \triangleright$  element count of
   tile  $T$ 
3: while some tile in  $\text{levels}[z]$  exceeds  $\text{TileCapacity}$  elements do
4:    $\text{next} \leftarrow$  empty  $2^{z+1} \times 2^{z+1}$  tile grid
5:   for all tiles  $T \in \text{levels}[z]$  do  $\triangleright$  per-tile inner loop
6:     split  $T$  into its four sub-tiles of  $\text{next}$   $\triangleright$  Section 3.2
7:      $\text{used} +=$  number of new elements added to  $\text{next}$ 
8:     if  $200 \cdot \text{used} > \mathcal{M}_{\max}$  then  $\triangleright$  memory budget; checked
       mid-split
       discard  $\text{next}$  and break pyramid growth
       end if
9:   end for
10:  end while
11:  if  $\text{next}$ 's tile width or height is below  $\text{minTileSize}$  then
12:    break  $\triangleright$  tile size below threshold
13:  end if
14:   $\text{levels}[z+1] \leftarrow \text{next}$ ;  $z \leftarrow z+1$ 
15: end while
16:  $Z \leftarrow z$ ;  $V_Z \leftarrow V(G)$ ;  $s_v^{(Z)} \leftarrow 1$  for all  $v \in V_Z$ 
17:  $V_{\text{PR}} \leftarrow$  nodes of  $G$  sorted by descending PageRank
18: for  $z = Z-1$  down to 0 do
19:    $(V_z, \{s_v^{(z)}\}) \leftarrow \text{SELECT\_TOPK\_WITH\_ADAP-}$ 
      $\text{TIVE\_SCALE}(V_{\text{PR}}, \lceil |V|/2^{Z-z} \rceil)$   $\triangleright$ 
     Section 3.1
20:   build a CDT from the inflated polylines of  $V_z$  at scales  $\{s_v^{(z)}\}$ 
21:   route every edge  $e \in E_z$  on this CDT
22:   rebuild the tiles of  $\text{levels}[z]$   $\triangleright$  Section 3.2
23: end for
24: return  $\text{levels}$ 

```

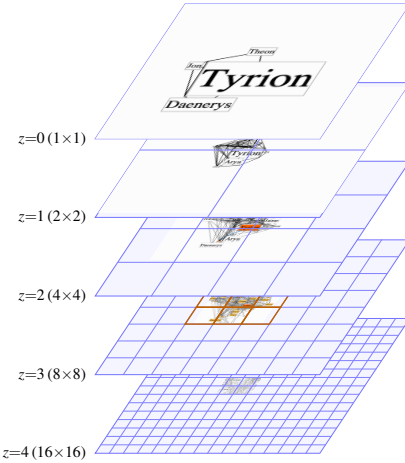


Figure 2: The entities rendered on each level are shown without scaling as the user zooms in.

2. the new tile width and height are both below ten times the average node width and height;
3. while generating the tiles of the new finest level, the running total of stored tile elements summed over all levels, multiplied by 200 bytes per element, exceeds the memory budget $\mathcal{M}_{\max}$, with default 4 GB; the partial level is then discarded and Z is set to the previous finest level.

The third condition is checked incrementally inside the level-build loop, so we abort as soon as the memory barrier is reached. On every benchmark graph in Table 1 the first condition fires; the second and third are defensive guards that bound depth on hypothetical inputs whose finest level cannot be packed within $\mathcal{C} = 500$.

Three phases. Once Z and the tile rectangles are fixed, each coarser level $z = Z-1, Z-2, \dots, 0$ is built in three phases:

1. pick the nodes of level z with PageRank and possibly enlarge them, Section 3.1;
2. reroute the edges of G_z around the resulting obstacles, Section 4;
3. recompute the per-tile edge clips on level z , Section 3.2.

Figure 1 shows two adjacent pyramid levels built by this procedure; Algorithm 1 summarizes the whole pipeline.

3.1 PageRank-Guided Level Construction

Before filling the level’s tiles for rendering we run PageRank [42] on G once, and sort all nodes of the graph, V , by descending rank. Write $k = Z - z$ for the level’s depth below the finest level. The candidates for a level of depth k are the nodes of the *prefix* of the sorted array of length $\lceil |V|/2^k \rceil$. The level’s edge set, used by Algorithm 1, is the subset of $E(G)$ whose endpoints both lie in this prefix.

Node positions are inherited from the original layout and never move: across levels we change only each node’s display size.

We add nodes to a level of depth k by walking the level’s prefix in rank-descending order. The top-ranked node is scaled by 2^k . We keep the node scales non-increasing, but as large as possible, while making sure that each next node does not overlap the previously selected nodes. If even at its original size the candidate’s bounding box would overlap an already-accepted node’s scaled box, the candidate is dropped.

Dropping a relatively high-ranked candidate at a coarse level is a deliberate compromise: we trade the inclusion of that candidate at this level for the invariant that the rendered nodes at every level remain pairwise disjoint. The cost is small in practice. Consider Arya’s node in the *Game of Thrones* graph: as Figure 2 shows, it is dropped from the top layer because a higher-ranked node, Tyrion, has covered the available region. The user still sees that part of the graph as occupied, and Arya’s individual node appears at a finer level where its box fits.

Spatial-hash overlap test. A naive overlap test against every previously accepted node would dominate the cost of the filter. We instead maintain the accepted boxes in a uniform spatial hash whose cell size is chosen so that any accepted box, and any candidate’s box at the level’s maximum scale, both fit inside one cell on each axis. Because of this choice, two such boxes can overlap only if their center cells differ by at most one step on each axis: testing a candidate therefore reduces to scanning the accepted boxes registered in the candidate’s own cell and in the eight cells immediately around it.

The hash brings little benefit at the coarsest levels, where the prefix is short and even a naive scan over the few accepted boxes is cheap; it pays off at finer levels, where the prefix grows geometrically and a per-candidate constant-time test replaces a linear scan over hundreds of already-accepted boxes. On the layouts we tested the expected work per candidate is constant, so the overall cost is dominated by the initial sort by PageRank. In our experiments this phase was never a bottleneck: the per-level routing dominates the build time at every level we measured.

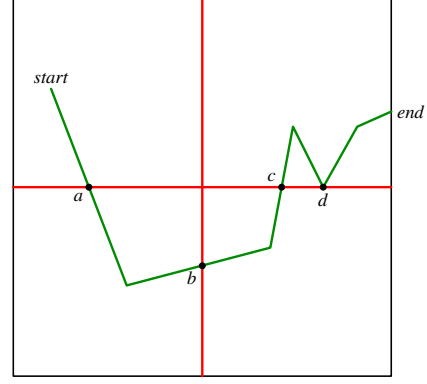


Figure 3: An edge clip, drawn as a polyline, from *start* to *end* inside a tile. The two red midlines split the tile into four sub-tiles and meet the polyline at the four points a, b, c, d ; at d the polyline touches the horizontal midline without crossing it. Cutting the polyline at these points yields five sub-clips $[start, a]$, $[a, b]$, $[b, c]$, $[c, d]$, $[d, end]$, each contained in one sub-tile and meeting that sub-tile’s boundary only at its endpoints.

3.2 Splitting Edges between Tiles

We describe edge routing later in Section 4, but for the purpose of this paragraph it is enough to know that each curve is a polyline.

The edge-clip generation works as follows. Consider a tile and an edge clip whose curve meets the tile boundary only at the curve’s endpoints. Let the *midlines* of a tile be the horizontal and vertical segments through its center; they split the tile into four sub-tiles. Cutting the curve with the midlines yields a set of sub-clips, each contained in one sub-tile, and each preserves the invariant: its curve meets the sub-tile boundary only at its endpoints. Because of this invariant, every split reuses the parent clip’s boundary crossings, and only the curve-vs-midline intersections need to be computed; no full curve-vs-rectangle clipping is required. Figure 3 illustrates one such split.

When we grow the pyramid in line 6 of Algorithm 1, each edge clip is already contained in its parent tile, so subdivision starts from that parent’s rectangle. When we prepare the tiles for rendering in line 23 of Algorithm 1, we have the rerouted graph G_z , so for each edge of G_z we start the subdivision from the top level rectangle.

Bundling edge clips. To reduce the number of elements rendered in a tile, all clips within a tile that share the same, or almost the same, pair of endpoints are bundled into a single clip whose attached edge list accumulates every contributing edge; the bundled geometry is taken from the first such clip encountered during splitting, and the choice is consistent within each level. Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

4 SLEEVE ROUTING ON THE CDT

To prepare for routing given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set of these polygons [17, 20]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. The weight of an edge is the Euclidean distance between the centroids of its triangles.

4.1 Routing inside a Sleeve

We split routing an edge into two subproblems: a *combinatorial* one—which sequence of triangles to cross—and a *geometric* one—which exact polyline through that sequence is shortest. The geometric step

Table 1: Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$; Z chosen by the natural stops of Section 3). The capacity $\mathcal{C}$ only guides the choice of Z ; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. “At level” is the level on which the maximum tile was found. In the worst cases, ca-HepPh and ca-CondMat, the densest tile holds 6–8 $\times$ the nominal capacity $\mathcal{C}$.

Graph	$ V $	$ E $	Levels	Max/tile	At level
gameofthrones	407	2 639	5	324	4
composers	3 405	13 832	8	749	3
ca-GrQc	5 242	28 968	8	359	4
ca-HepTh	9 877	51 946	8	1 836	3
facebook_combined	4 039	88 234	9	928	5
ca-HepPh	12 008	236 978	9	3 643	3
ca-CondMat	23 133	186 878	8	2 949	3
deezer_europe	28 281	92 752	8	2 402	3
delaunay_n15	32 768	98 274	8	283	7

is the classical funnel algorithm [16, 30], which pulls the path taut inside the polygon formed by the sleeve triangles.

To obtain the sleeve we run a shortest-path search on D , where triangles inside obstacles other than the source and target obstacles are excluded. The natural per-edge instantiation is A^* [29] with the straight-line distance to t as heuristic. When a node has many incident edges, however, running independent searches is wasteful. We instead group edges by source s and run a single Dijkstra computation per source, expanding until all targets are reached and recovering each sleeve by following parent pointers. This gives a speedup over A^* ; for example on Game of Thrones, with 407 nodes and 2 639 edges, it cuts routing time by $\sim 23\%$. Table 3 reports the gain across the full benchmark suite.

We can improve even more. Since the query edges are undirected on D , each can be served from either endpoint, so the number of distinct roots is in turn reducible by reorienting query edges to minimize that count. Let the *demand graph* H be the graph whose vertices are the graph nodes and whose edges are the query edges still to be routed. A set of roots R is feasible iff every demand edge has at least one endpoint in R , that is, R is a vertex cover of H , and the smallest such R is a minimum vertex cover of H . The problem of finding a minimum vertex cover is NP-hard [36], so we approximate it with the standard greedy maximum-degree rule: while H still has an edge, pick a vertex of maximum degree in H , add it to R , and delete it together with its incident edges. Implemented with degree buckets, the procedure runs in linear time. Each demand edge is then routed from whichever endpoint lies in R . To our knowledge no prior edge router exploits a vertex cover of the demand graph in this way; many-to-many shortest-path frameworks [37] also amortize per-source searches, but they take the source set as given and do not reduce it through a covering argument. Applying this heuristic reduces the number of Dijkstra trees on Game of Thrones from 331, one per distinct source, to 198, and on the composers graph with 3 405 nodes and 13 832 edges from 2 645 to 1 281, shortening the routing pass by $\sim 28\%$ and $\sim 31\%$ respectively.

In addition, before launching the sleeve search for an edge we try routing the straight line segment between the source and the target by walking the CDT.

Route quality. To assess route quality, we compared the sleeve router with the optimum on the full visibility graph of the padded obstacles, on the Game of Thrones layout. The visibility-graph optimum is the shortest taut polyline around the obstacles; we computed it by all-pairs Dijkstra over the polygon corners and the node centers. This is a lower bound on any obstacle-avoiding route. On the 407-node, 2,639-edge layout the sleeve router has total arc length

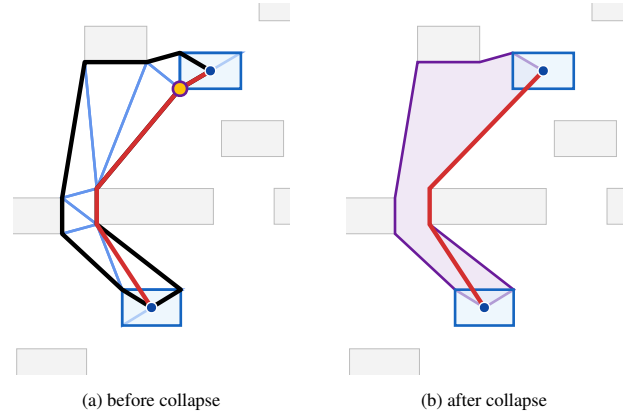


Figure 4: Effect of vertex collapse on the LORAS-RENTLY edge of the Game of Thrones graph. Endpoints are shown as blue dots; their padded obstacles are highlighted in light blue. (a) The CDT triangles forming the sleeve are drawn in cornflower blue and the funnel path in red; the thick black polyline is the perimeter of the non-collapsed sleeve. The yellow dot marks the source-obstacle vertex that gets collapsed. (b) The sleeve after collapse, drawn as a single purple polygon with the source/target obstacles merged into the corresponding endpoints, and the funnel path (red) through it. Collapsing widens the funnel opening at the endpoint and removes the spurious detour around the obstacle corner.

714.3 k arc-length units against 695.1 k for the visibility-graph optimum, a ratio of $1.028\times$. The worst per-edge ratio is 1.37, much less than the constrained Delaunay triangulation’s worst-case stretch $4\pi\sqrt{3}/9 \approx 2.42$ [14].

Edge hint. We explored a routing variant in which the path generated for an edge on one level provides a hint for routing the same edge on an adjacent level, increasing the visual stability of the routes between levels. Its shortcoming is that the search cannot be batched per source as the Dijkstra-tree variant above is, which impacts performance.

4.2 Vertex Collapse at Source and Target

We improve the sleeve’s geometry in the situation depicted in Figure 4. The remedy is to *collapse* corner-hugging chain vertices to the corresponding endpoint. Walking the right chain of the sleeve outward from s , we find the first vertex of the source obstacle at which the chain makes a right turn, and replace every source-owned vertex on that chain up to and including this one by s . The left chain is processed mirror-symmetrically, with left turns instead of right, and the target end likewise. Diagonals whose two endpoints coincide after collapse are dropped, and the left/right orientation of each surviving diagonal is preserved from the uncollapsed geometry. The collapsed vertex in Figure 4(a) is marked in yellow.

After collapse, the funnel sees a wide opening at each endpoint, as in Figure 4(b), and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves. Collapse might cause the resulting path to overlap nodes other than the source and the target, but we have found the effect to be negligible.

The implementation also offers an optional pass that fits a Bezier segment into each polyline corner to render edges as smooth curves. It is disabled by default for performance: the polyline rendering is fast and visually acceptable, so we trade the smoother curves for routing speed.

5 RENDERING WITH DECK.GL

The output of the build pipeline of Section 3 is consumed in the browser by the `TileLayer` of `@deck.gl/geo-layers` [10]. `TileLayer` is the standard component used by online tile-map renderers: it tracks the viewport, determines which tiles are needed at the current zoom, and asynchronously requests them through a user-supplied `getTileData` callback, caching the results across frames so that subsequent pans within the cache require no further work.

GJS integrates with this component at construction time by exposing the precomputed pyramid as the `TileLayer`’s zoom range, with the coarsest pyramid level mapped to the layer’s lowest zoom and the finest level to the highest. As the user pans and zooms, `TileLayer` resolves the viewport to an integer zoom and a set of axis-aligned tile indices and, for each tile it needs to display, calls `getTileData`. The renderer translates each such request back into a tile of the corresponding pyramid level and returns its precomputed nodes, edge clips, edge labels, and arrowheads. The mapping is one-to-one, so no geometric work is performed at view time: panning shifts the visible window over the tile cache, and zooming displays tiles from the adjacent level using `TileLayer`’s built-in cross-fade.

The rendered geometry depends only on the zoom level and does not change with the viewport, except for the clipping to the visible window. Within each level, the displayed nodes are the ones chosen for that level by the construction of Section 3.1, and the displayed edges are the geodesics around those nodes, computed by the sleeve router of Section 4.

Interaction. The renderer supports pan, zoom, and hover highlighting of nodes and edges. Hovering an edge highlights it and its incident nodes; hovering a node highlights the node, its one-hop neighbors, and its two-hop neighbors, each in a different color. A `zoomTo` API animates the viewport to a target rectangle with a smooth transition and is used, for example, by the search box to focus the view on a query result.

6 EXPERIMENTAL RESULTS

We evaluate the sleeve router and the tile-pyramid build on the same benchmark graphs as Table 1: the Game of Thrones social network [13], the GD 2011 contest `composers` graph [9], four SNAP collaboration networks (`ca-GrQc`, `ca-HepTh`, `ca-HepPh`, `ca-CondMat`) and the SNAP `facebook.combined` ego network [39, 3], the `deezer.europe` social network [44], and the SuiteSparse Delaunay mesh `delaunay.n15`. All experiments run in headless Chrome 148 on an Apple M4 Max laptop with 48 GB of unified memory, with the unmodified browser bundle of GJS loaded into the page. Layout uses IPSep-CoLa, and the tile pyramid is built with the same parameters as Table 1: capacity $\mathcal{C} = 500$, with Z chosen by the natural stops of Section 3.

Table 2 reports per-graph timings of three pipeline stages: *Routing*, the sum of the two phases logged by the sleeve router on the full graph—constrained Delaunay triangulation of the obstacles and the Dijkstra-tree search on the CDT dual, Section 4.1; *Tiling*, the build of the tile pyramid, Section 3, which includes the per-level routing; and *Total*, the end-to-end loading time including parsing.

The CDT phase contributes negligibly to the routing column—from 6 ms on Game of Thrones to about half a second on the largest graphs—so virtually all of *Routing* is the Dijkstra-tree sleeve search, with the vertex-cover heuristic of Section 4.1 reducing the number of trees we run. Even on graphs with hundreds of thousands of edges (`ca-CondMat`, `ca-HepPh`, `deezer.europe`) the full pipeline finishes in a few minutes; this is a one-time build cost. The resulting tile pyramid already contains the per-level routes, so panning and zooming only fetch and render the precomputed tiles, with no rerouting at view time.

Table 3 isolates the contribution of the two source-side optimizations of Section 4.1: batched Dijkstra trees over per-edge A^* and the vertex-cover heuristic on top of that. Each row times a single

Table 2: End-to-end loading benchmarks in headless Chrome 148 on an Apple M4 Max laptop, all times in seconds. Measurements come from the `chrome-routing-bench` example, which runs the unmodified browser bundle of GJS. *Routing* sums the CDT-construction and Dijkstra-tree sleeve-search phases of Section 4.1 on the full graph; *Tiling* is the build of the tile pyramid, capacity $\mathcal{C} = 500$ with Z chosen by the natural stops of Section 3, which includes the per-level routing; *Total* is the total loading time including parsing of the graph file, graph layout, and all other stages.

Graph	$ V $	$ E $	Routing	Tiling	Total
<code>gameofthrones</code>	407	2 639	0.07	0.07	0.38
<code>composers</code>	3 405	13 832	1.49	0.74	2.69
<code>ca-GrQc</code>	5 242	28 968	1.78	0.87	3.40
<code>facebook.combined</code>	4 039	88 234	4.34	2.36	7.77
<code>ca-HepTh</code>	9 877	51 946	9.33	3.41	14.87
<code>ca-HepPh</code>	12 008	236 978	38.89	14.08	57.71
<code>ca-CondMat</code>	23 133	186 878	79.72	20.52	110.06
<code>deezer.europe</code>	28 281	92 752	134.78	23.09	172.68
<code>delaunay.n15</code>	32 768	98 274	18.71	5.64	41.38

sleeve-search pass run after layout, with the constrained Delaunay triangulation excluded so that only the search costs are compared. The batched Dijkstra trees outperform per-edge A^* on every social graph, with the speedup growing with the average degree and reaching +63 % on `facebook.combined` and +48 % on `ca-HepPh`; the vertex-cover heuristic yields a further 11–42 % reduction on the same graphs. The Delaunay mesh `delaunay.n15` is the only outlier: the degrees of the nodes are small and the shortest paths do not extensively overlap, so neither optimization has a chance to improve performance.

Browsing smoothness. We test how smoothly a user can browse a freshly loaded graph in the browser. On the same M4 Max laptop as Tables 2 and 3, a script loads a graph, waits for the tile pyramid to build, and then plays back twelve dives. Each dive picks a random point inside the laid-out graph, recenters the view on that point, zooms from the coarsest view all the way in to the finest detail in one second, then zooms back out to the coarsest view in another second. The twelve random points are independent and the dives run back to back.

While the dives play, the script measures three things on the browser side. First, how often the screen actually refreshes: at 60 Hz the screen refreshes every 16.7 ms, and a longer gap is visible as a stutter. We report the 95th percentile of the gap between successive refreshes, the value below which 95 % of refreshes fall. Second, how often the browser pauses for more than 50 ms on a single block of work; this is the standard browser metric of a “long task” and each one freezes the page. Third, at the deepest point of every dive, how many graphical objects the browser must draw, that is, nodes, edge clips, labels, and arrowheads, summed over the tiles that fall inside the viewport. This last number is a proxy for how hard the GPU has to work each frame. Table 4 reports the result.

Across all nine graphs the dives stay close to the 60 Hz target. On every graph, 95 % of the gaps between successive refreshes are at most 18.4 ms, only 1.7 ms over the ideal 16.7 ms gap at 60 Hz and too small to be perceived as a stutter. The browser pauses for more than 50 ms only a handful of times across the twelve dives on the three densest graphs, `ca-HepPh`, `ca-CondMat`, and `deezer.europe`, and not at all on the smaller ones, so the moments of stutter that do happen are short and rare. At the deepest zoom, the browser is asked to draw at most 2567 objects on `gameofthrones` and at most 2505 on `deezer.europe` even though the latter graph has 35 times more edges. This is the property the pyramid is designed to deliver: how hard the GPU has to work each frame depends on what is currently on screen, not on the total size of the graph.

Table 3: Sleeve-search times of the three routing modes on the benchmark graphs of Table 2, in headless Chrome 148. Columns: *Srcs*, the number of distinct edge endpoints in the demand graph—one per source for the per-edge A* baseline; *VC*, the number of Dijkstra trees actually launched after the greedy maximum-degree vertex-cover reorientation of Section 4.1; *A**, wall-clock seconds of one full sleeve-search pass with per-edge A* (straight-line distance to the target as heuristic); *DJ*, wall-clock seconds with the batched Dijkstra-tree mode—one tree per source, expanding until all of that source’s targets are reached; *VC.DJ*, wall-clock seconds with the vertex-cover-reoriented Dijkstra-tree mode—same as DJ but with only *VC* roots; $DJ/A^* = (A^* - DJ)/A^*$, speedup of DJ over A*; $VC/DJ = (DJ - VC.DJ)/DJ$, additional speedup of VC.DJ over DJ. The CDT-construction phase, identical across modes, is excluded so that only the search costs are compared.

Graph	Srcs	VC	Time (s)			Speedup	
			A*	DJ	VC.DJ	DJ/A*	VC/DJ
gameofthrones	331	198	0.077	0.059	0.043	+23 %	+28 %
composers	2 645	1 281	2.60	2.14	1.49	+18 %	+31 %
ca-GrQc	5 241	2 796	2.55	2.52	1.65	+1 %	+35 %
facebook_combined	3 663	3 045	13.31	4.88	4.36	+63 %	+11 %
ca-HepTh	9 875	5 003	17.13	15.42	9.28	+10 %	+40 %
ca-HepPh	12 006	7 032	111.2	58.07	36.38	+48 %	+37 %
ca-CondMat	23 133	13 564	189.1	131.1	76.45	+31 %	+42 %
deezer_europe	21 060	13 450	172.1	154.0	124.4	+11 %	+19 %
delaunay_n15	28 467	23 456	13.57	16.02	14.88	−18 %	+7 %

7 CONCLUSION

We presented GJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The sleeve routing method routes edges directly on the CDT dual graph using batched Dijkstra trees and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

Future work. The current approach builds the entire tile pyramid up front, which is expensive and bounds the size of graphs we can handle. A natural next step is to build only the first few levels quickly and hand them to the renderer; the remaining levels would be built lazily as the user browses the graph. This would lower the time-to-first-frame and lift the limit on graph size.

A second challenge is that some tiles can contain many more elements than the capacity parameter $\mathcal{C}$ that guides the build. Table 1 reports densest tiles holding 6–8 $\times$ the nominal $\mathcal{C}$ in the worst cases, which leaves the rendering and interaction cost uneven across the viewport. Enforcing an actual per-tile budget remains an open problem.

SUPPLEMENTAL MATERIALS

GJS is available as open source; an anonymized mirror of the repository is provided for review at <https://anonymous.4open.science/r/gjs-163F/>. A live demo of the WebGL renderer with sleeve routing is available at <https://gjs-sleeve.surge.sh/>. The benchmark drivers used for Tables 1–4 ship with the repository.

REFERENCES

- [1] Cosmograph. <https://cosmograph.app>. 1
- [2] deck.gl: Large-scale WebGL-powered data visualization. <https://deck.gl/>. 1
- [3] facebookcombined. https://snap.stanford.edu/data/facebook_combined.txt.gz. 5
- [4] Funnel algorithm. <https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf>. 2
- [5] Graphviz. <http://www.graphviz.org/>. 1

Table 4: Browsing-smoothness measurements on the benchmark graphs. For each graph the script plays back twelve dives; each dive picks a random point inside the laid-out graph, recenters on it, and zooms from the coarsest view to the finest and back in two seconds. *Z*, number of tile-pyramid levels built; *LT*, number of main-thread pauses longer than 50 ms observed during the twelve dives; *LT total*, the total duration of those pauses, in milliseconds; *Frame p95*, the 95th percentile of the gap between successive on-screen refreshes during the twelve dives, in milliseconds; *Elts max*, the largest number of graphical objects, that is, nodes, edge clips, labels, and arrowheads, that the browser had to draw at the deepest point of any dive.

Graph	V	E	Z	LT	LT total	Frame p95	Elts max
gameofthrones	407	2 639	5	0	0	17.4	2 567
composers	3 405	13 832	9	1	51	18.2	497
ca-GrQc	5 242	28 968	9	0	0	18.2	677
ca-HepTh	9 877	51 946	9	0	0	18.2	383
facebook_combined	4 039	88 234	9	0	0	18.0	495
ca-HepPh	12 008	236 978	9	9	556	18.4	1 394
ca-CondMat	23 133	186 878	9	5	273	18.2	1 391
deezer_europe	28 281	92 752	9	3	164	18.4	2 505
delaunay_n15	32 768	98 274	9	0	0	18.1	1 012

- [6] Regraph. <https://cambridge-intelligence.com/regraph/>. 1
- [7] sfdp. <https://graphviz.org/docs/layouts/sfdp/>. 1
- [8] Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. <https://www.sigmapjs.org/>. 1
- [9] Skewed. <http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml>. 5
- [10] TileLayer (@deck.gl/geo-layers). <https://deck.gl/docs/api-reference/geo-layers/tile-layer>. Accessed: 2026. 5
- [11] yworks. <https://yworks.com/products/yed.2>
- [12] J. Abello, F. Van Ham, and N. Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, vol. 12, pp. 669–676. IEEE, 2006. 2
- [13] A. Beveridge and M. Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pp. 201–225. Routledge, 2018. 5
- [14] P. Bose and J. M. Keil. On the stretch factor of the constrained Delaunay triangulation. In *3rd International Symposium on Voronoi Diagrams in Science and Engineering (ISVD)*, pp. 25–31. IEEE, 2006. doi: 10.1109/ISVD.2006.7 4
- [15] U. Brandes and C. Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18–20, 2006. Revised Papers 14*, pp. 42–53. Springer, 2007. 1
- [16] B. Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pp. 339–349. IEEE, 1982. 2, 4
- [17] B. Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennykh Nauk*, 7(1):793–800, 1934. 3
- [18] D. Demyen and M. Buro. Efficient triangulation-based pathfinding. In *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI)*, pp. 942–947, 2006. 2
- [19] D. P. Dobkin, E. R. Gansner, E. Koutsofios, and S. C. North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD’97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pp. 262–271. Springer, 1997. 2
- [20] V. Domiter and B. Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008. 3
- [21] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. <https://arxiv.org/abs/2504.17033>. 2
- [22] T. Dwyer, Y. Koren, and K. Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions*

- on Visualization and Computer Graphics, 12(5):821–828, 2006. doi: 10.1109/TVCG.2006.156 1
- [23] T. Dwyer and L. Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pp. 147–158. Springer, 2010. 2
- [24] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016. 1
- [25] E. R. Gansner, Y. Koren, and S. North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005. 2
- [26] E. R. Gansner and S. C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000. 1
- [27] L. Guibas, J. Hershberger, D. Leven, M. Sharir, and R. E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987. 2
- [28] D. Harel and Y. Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, vol. 6, pp. 179–202, 2002. 2
- [29] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136 4
- [30] J. Hershberger and J. Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994. 4
- [31] J. Hershberger and S. Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999. 2
- [32] D. Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006. 2
- [33] D. Holten and J. J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, vol. 28, pp. 983–990. Wiley Online Library, 2009. 2
- [34] Y. Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005. 2
- [35] C. Hurter, O. Ersoy, and A. Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, vol. 31, pp. 865–874. Wiley Online Library, 2012. 2
- [36] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, eds., *Complexity of Computer Computations*, pp. 85–103. Plenum Press, New York, 1972. 4
- [37] S. Knopp, P. Sanders, D. Schultes, F. Schieferdecker, and D. Wagner. Computing many-to-many shortest paths using highway hierarchies. In *Proceedings of the 9th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pp. 36–45. SIAM, 2007. 4
- [38] D. T. Lee and F. P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984. 2
- [39] J. Leskovec, J. Kleinberg, and C. Faloutsos. Graph evolution: Densefication and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007. 5
- [40] L. Nachmanson, R. Prutkin, B. Lee, N. H. Riche, A. E. Holroyd, and X. Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pp. 3–15. Springer, 2015. 1
- [41] L. Nachmanson, G. G. Robertson, and B. Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pp. 389–394. Springer, 2008. 1
- [42] L. Page, S. Brin, R. Motwani, and T. Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999. 3
- [43] A. Perrot and D. Auber. Cornac: Tackling huge graph visualization with big data infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018. 2
- [44] B. Rozemberczki and R. Sarkar. Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, p. 1325–1334. ACM, 2020. 5
- [45] J. R. Shewchuk. Delaunay refinement algorithms for triangular mesh generation. *Computational Geometry*, 22(1–3):21–74, 2002. 2
- [46] K. Sugiyama, S. Tagawa, and M. Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi: 10.1109/TSMC.1981.4308636 1
- [47] M. Wybrow, K. Marriott, and P. J. Stuckey. Orthogonal connector routing. *Lecture Notes in Computer Science*, 5849:219–231, 2010. 2